CQM Case: Future Sustainable Logistic Network

EBC 2107 - Advanced Algorithms
Maastricht University
School of Business and Economics

April 4th 2025

Aymeric van Ysseldijk, Maud te Braak, and Moos Vos
(Student IDs: i6316020, i6326454, i6331214)

Introduction

We live in an age with unprecedented levels of globalisation and a world where environmental efforts and sustainability come to the foreground increasingly more frequent. In line with these global developments the Greenforwarder recognizes it is time to adapt their strategy. They want to radically revise their network and have asked for our help to help them adapt to the current trends to skyrocket their growth for the future.

The Greenforwarder has set us up with the task of finding the best network of connections and modalities to use for these specific connections to optimize their operations. To quantify this task they have set up 3 KPI's to focus on: Profit, Emissions and Reach. Profit is defined as follows:

$$Profit = TotalRevenue - TotalInvestment$$

The cost of investment is summarized in the following table:

Additional Capacity	**Expense (Cumulative)**			
(Cumulative)	**Truck**	**Ship**	**Train**	**Airplane**
5 (5)	3 (3)	1 (1)	1 (1)	1 (1)
5 (10)	3 (6)	2 (3)	1 (2)	3 (4)
5 (15)	3 (9)	3 (6)	2 (4)	5 (9)
5 (20)	3 (12)	4 (10)	4 (8)	7 (16)
5 (25)	3 (15)	5 (15)	8 (16)	9 (25)

Table 1: Investment costs

For emissions, we received the following table:

Modality	**Emissions**
Truck	8
Airplane	25
Ship	1
Train	3

Table 2: Emissions for modalities

Then finally regarding reach, the world is split up into 10 distinct regions. The Greenforwarder wants to be present in as many of these regions as possible. This is why the reach is defined as the total amount of regions connected in the network.
To quantify these KPI's into one expression we have the following formula:

$$Score = w_p \cdot P - w_E \cdot E + w_R \cdot R$$

The Greenforwarder has tasked us to come up with solutions for three different problems. Firstly the POC problem, to explore whether it is possible to reach out to a great amount

of regions while still keeping emissions limited. To test this we the following weights to take into account: $w_E = 1$ and $w_R = 100$.

The next problem is the MVP scenario, where we start to take into account the profit as well. For this we get the following objective weights: $w_p = 15$, $w_E = 1$, and $w_R = 100$.

For our final scenario called Live, we get the restriction of having a profit that is at least 45. In the end Greenforwarder needs to make money to grow the organisation so profit is needed to make this reality. The objective weights do not change and are still: $w_p = 15$, $w_E = 1$, and $w_R = 100$.

In this paper we will first explain the general functioning of our algorithm with detailed descriptions of the process and the thought process behind the ideas. To understand the algorithm it is important to understand the working of Prim's algorithm where the algorithm is inspired on.

In theory Prim's algorithm is a greedy algorithm that is used to find the Minimum Spanning Tree of an undirected graph. The algorithm starts with a vertex and expands the spanning tree by adding the lowest weight edge from the neighbouring set of that vertex. It then continues with the updated neighbouring set and stops until all vertices are connected. The algorithm is written out below.

Algorithm 1: Prim's algorithm

Input: $G = (V, E, w)$;
$T = \emptyset, W = \{v\}$ for some $v \in V$
Output: Minimum Spanning Tree;
while $W \neq V$ **do**
  find $e = \{v, w\} \in \delta(W)$ s.t. $c_e \leq c_f \forall f \in \delta(W)$
  $T = T \cup \{e\}$
  $W = W \cup \{v, w\}$ end-while
Here $\delta(W) = \{\{v, w\} \in E : w \in W, v \in V \setminus W\}$, for $W \subset V$
Return: T

After explaining our general model we will dive into the adaptations that are necessary to optimize the algorithm for the different situations. Continuing we will look into the results, we will do a sensitivity analysis accompanied by the results for the scenarios posed by the Greenforwarder. Then finally we will get into our conclusion addressing the implications of our results, discussing the limitations of the algorithm and recommendations both for the actionsteps of the Greenforwarder and for improvements of our model.

Description of Our Algorithm

To start off our paper we will explain the functioning of our algorithm. As all the connected cities need to be reachable from Eindhoven - Greenforwarders base of operations - we initialize the tree from Eindhoven. We then compute the score of all the edges in the neighbouring set, for all the different modalities available per edge. We compute this score based on the previous shown formula for score:

$$Score = w_p \cdot P - w_E \cdot E + w_R \cdot R$$

Here profit is only accounted for in the calculation of the edge score if both cities of a target are in the network. An example of how our algorithm would work is illustrated below:

Figure 1: Illustration of the functioning of the algorithm

Our algorithm then picks the best score edge and accompanied modality. In the case above we have an equal best score edge with London and Paris having the same score of -12, in such a case our algorithm picks one of the connections arbitrarily. In our case it picked London adding it to our tree and generating our new neighbouring set as shown below.

Figure 2: Example continued

After every iteration the network score is saved, the algorithm keeps on running until all modalities have run out, including those under investment, and finally the algorithm outputs the iteration at which the network score was the highest. So the result of the algorithm is not where it stops adding edges, but on what iteration of the algorithm the network score was highest.
This is how our general algorithm works for the POC scenario we did not implement any adaptations.

MVP Adaptations

For the MVP scenario we do have to consider the profit so we will be looking at targets. However while we were looking at the output of our algorithm we realised we needed to make an adaptation. Because when determining the score of an edge we were only increasing its score for profit when the second target gets reached. Although this makes sense it does imply that our algorithm does not actively search for and connect to first targets resulting in less targets being reached in the long term.

To solve this problem we introduced a separate edge scoring system. With the following formula:

$$EdgeScore = w_p \cdot (P + Targetbonus) - w_E \cdot E + w_R \cdot R$$

As you can see we have added a Targetbonus in this scoring system. We basically increase the score of an edge if the node is part of a target, to motivate the algorithm to make connections to as many targets as possible. To illustrate this there is an example below:

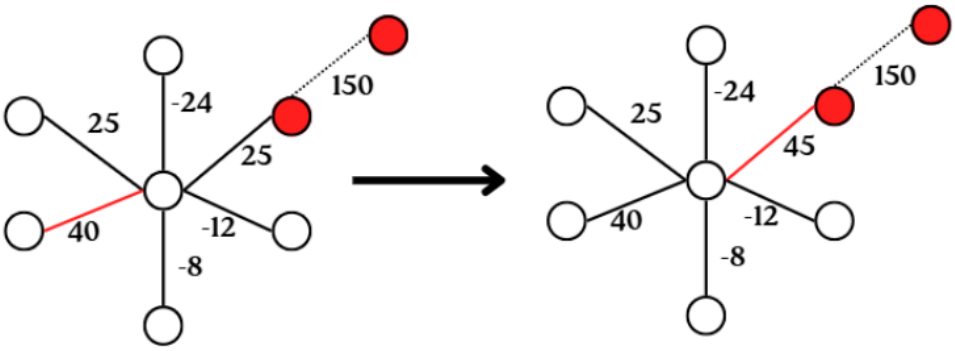

Figure 3: Example: Effect TargetBonus

The algorithm would initially pick the edge of score 40, however choosing the edge of score 25 could result in a higher profit in the next iteration of the algorithm. When implementing the TargetBonus, the algorithm now does pick the edge of score 45, and in the next iteration the edge of 150 will be choosen. This adaptation increased the score of the network significantly.

Live Adaptations

Now for our Live scenario we tried to run the same model as in our MVP scenario but we did not achieve the lower bound for profit of 45. We had to find a way to motivate the algorithm to prioritize profit. To do this, we introduced the ProfitBooster. The ProfitBooster was again a measure to adapt the edge score to our specific problem. The formula is as follows:

$$EdgeScore = w_p \cdot (P \cdot ProfitBooster + TargetBonus) - w_E \cdot E + w_R \cdot R$$

Using this formula we were able to motivate the program to achieve the profit lower bound. While this was successful we did end up using many planes to achieve it, resulting in incredibly high emissions and a massively negative score.
Upon analysing our output we discovered that our algorithm logically used all the low emission modalities first which resulted in the necessity to use planes later on for the

very high emission edges. To solve this issue and motivate our program to use planes earlier on for the lower emission edges, we installed a constraint. If the best score in the neighbouring set is below a certain threshold we force the program to use an airplane as these low score edges would always be short connections that don't take up a lot of emission units. In this way we forced our algorithm to save the low modalities for higher emission use edges later. For illustrative purposes we have added an example below.

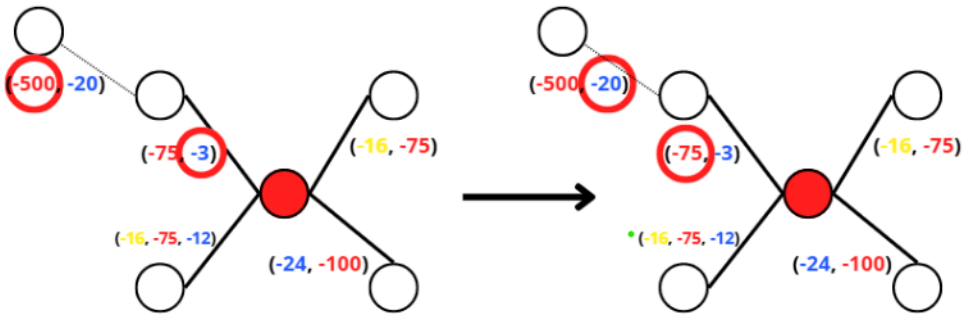

Figure 4: Example effect saving low emission modalities

In this example the threshold value is 0 as you can see all of the options are below so we pick the option for plane instead. Enabling us to pick the boat option later on!

Results

For the portrayal of the results we will add the map of connections and a table with the results regarding: Score, Investment, Profit, Emissions and Reach.

POC Scenario

Metric	Results POC
Score	635
Investment	10
Profit	-10
Emissions	265
Reach	9

Figure 5: Results POC Scenario

In the POC scenario we were tasked to get into as many regions as possible while keeping emissions low. In our result, the algorithm therefore only picks train and ship modalities as these have the lowest emissions. Also the algorithm manages to get into 9/10 of the regions.

MVP Scenario

Metric	Results MVP
Score	678
Investment	6
Profit	14
Emissions	432
Reach	9

Figure 6: Results MVP Scenario

In our MVP solution you can see some extra connections being made to achieve targets. These are illustrated by the coloured nodes on the map.

Live Scenario

Metric	Results Live
Score	619
Investment	3
Profit	47
Emissions	886
Reach	8

Figure 7: Results Live Scenario

In these results from the Live scenario we have illustrated the amount of needed units of modality by the thickness of the lines. As you can see the airplane lines are all very thin, and in fact we never use more than 5 units of plane modality for a connection showing the effectiveness of our method.

After adding these adaptations we decided to implement them back into the MVP model as well to see if it had an effect, and it actually improved our results! You can see them here below.

Improved MVP Scenario

Metric	Results MVP
Score	713
Investment	0
Profit	20
Emissions	387
Reach	8

Figure 8: Results improved MVP Scenario

We see an improvement in the score here as the algorithm gave a higher priority to profit, sending the algorithm in a different direction. So not only did this improve profit, it also improved our score.

Conclusion

In conclusion, our algorithm produces an optimised and radically revised plan for the network that the Greenforwarder should build. As our algorithm is especially strong at keeping down emissions at low levels of profit we would advise the Greenforwarder to invest in their environmental image globally. Using our algorithm you can have the power to truly show the steps that you are making towards making the shipping industry a greener sector, and building on this will only play into your favor for the future.

Not only is our algorithm useful for this purpose. It is also very flexible and easy to use. If the Greenforwarder for example realizes they want to put a greater focus on their profits, you can simply adjust the Targetbonus and Profitbooster variables in the model and it will create a new network within seconds that prioritizes this profit to a greater extent, which can be read below in the sensitivity analysis. We recommend the Greenforwarder to let us teach their employees to use this model for future changes in policy that might be necessary.

On the other hand, our model does have its limitations. The mayor limitation is its incapability to look into the future. In fact our algorithm is a greedy algorithm that simply picks the best modalities and edges now without thinking about the future downsides that choice might have. Of course we have put some redirecting measures into place but it continues to be a problem in our algorithm.

Another problem is the lack of reflection. As soon as the algorithm has used all the modalities it outputs the best iteration that we found. However, in this iteration we oftentimes find edges that could be better removed as they simply function as leaves of the tree carrying a negative score for the network. Removal of these leaves would improve our solutions.

This leads right into the future improvements we can do. Firstly we can partly solve the problem of looking into the future by letting the problem look 2 steps into the future. Where it looks at all the options in the neighbouring set and towards all the options in the neighbouring set of the neighbouring set. To see which combination of 2 edges would yield the highest score. To then continue only adding the first edge in this combination and repeat this process. Of course we could also increase this 2 steps to 3 or 4 steps into the future depending on how much our computers can take. In this way our algorithm will be able to make a more educated decision on the edges that need to be added to the network.

A solution for our other problem would be to do a reflection check on the outputted graph. Where we remove all the edges that have a negative impact on our score whilst still keeping the graph connected.

To round off this conclusion, our algorithm is a very fast running, easy to understand and flexible model. Although the results that we found so far are very respectable, we

know that these can be improved to a great extent, and continuously adapted to whatever the Greenforwarder is looking for. Choosing to work with our algorithm is investing in an environmentally progressive and flexible future.

Sensitivity Analysis

For the MVP and Live scenario, 3 main adaptations were used. We are manipulating the algorithm by changing the edge scores to have it make different choices in each iteration. By changing the values of the ProfitBooster, TargetBonus and the treshold of favoring planes, different results will follow. For all three variables, a loop was created to let the variable range over values and output for which value the variable has the highest network score.
The best ProfitBooster has a value of 2. However, changing the goal to maximizing profit while maintaining a positive network score, a ProfitBooster of 1.3 gives a profit of 61 with a total network score above 200. The same is done for the TargetBonus, where the best value is 2.2 with a network score of 619. For the threshold of favoring planes, the best threshold value is 0 with also a network score of 619.

Increasing or decreasing the variables makes the network score change. In the table below, the impact of the changes of the variables on the network score can be seen.

	1.0	2.0	100
ProfitBooster	-335	619	619
	1.5	2.2	100
TargetBonus	611	619	522
	0	2	10
Plane Threshold	619	393	237